

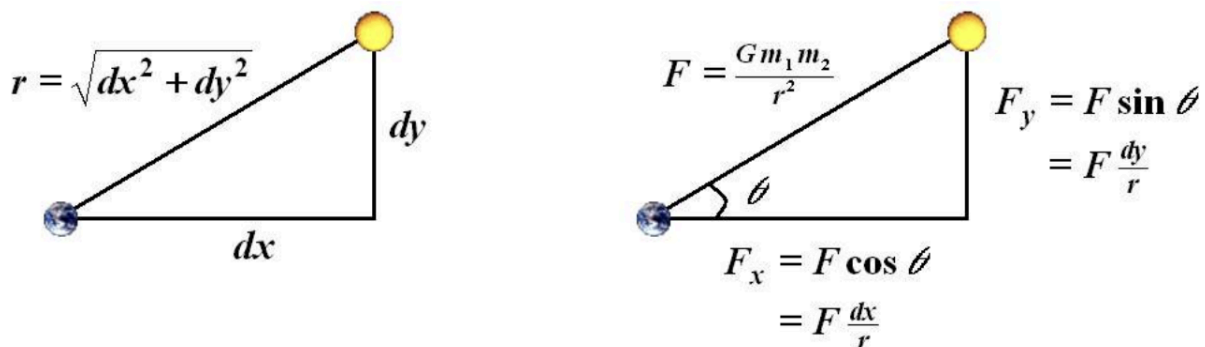
Summary

I implemented Barnes-Hut for N-body gravity simulation in CUDA on a GPU. My best implementation obtains an $\sim 423\times$ speedup compared to the brute force $O(N^2)$ gravity algorithm on GPU and an $\sim 800\times$ speedup compared to single thread Barnes-hut CPU implementation on 5 million bodies with less than a 0.1% average force error rate. My implementation follows the ideas laid out in [1]. No starter code was used for this project. My code can be found at <https://github.com/Andrew-Koulogeorge/barnes-hut>

Introduction

Simulating the gravitational interactions among large collections of bodies across many time steps is a classical problem in computational physics. Given N bodies, each exerting a gravitational force on every other, we wish to compute the net gravitational force acting on each body. The naive approach is to evaluate every pairwise interaction directly. For N bodies, this requires $O(N^2)$ force computations per time step — a calculation that quickly becomes intractable as N grows. Simulating one million stars would require on the order of trillions of interaction evaluations per step.

The key insight to make large-scale simulation feasible comes from the formula for gravitational force. As a review from high school physics, the gravitational force between two bodies is proportional to the mass of each of the bodies and the squared distance between them.



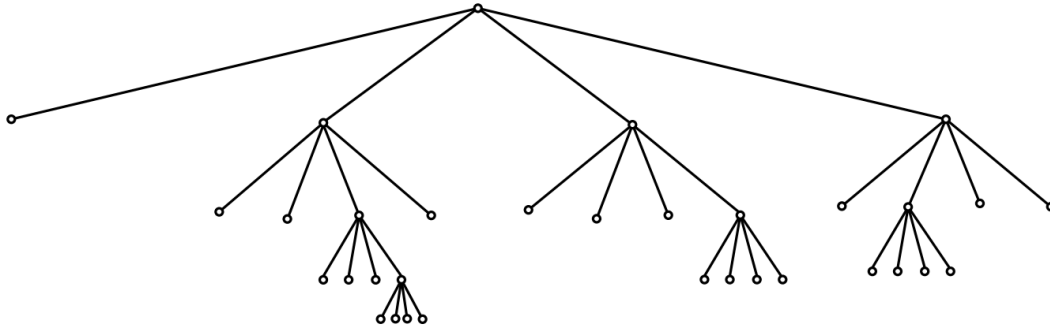
Gravity decays with the inverse square of the distance between two bodies. This suggests a natural approximation: rather than computing the exact contribution of every distant body individually, one can treat a cluster of distant bodies as a single mass located at the cluster's

center of gravity. While this introduces error, at sufficiently large distances, the cluster's internal structure has negligible effect on the force it exerts.

The challenge then becomes implementing an efficient partitioning of our space of bodies and deciding when a cluster is "far enough" to be treated as a single entity. The Barnes-Hut algorithm answers these questions by recursively decomposing space into an OctTree (or QuadTree in 2D), enabling force computations to be performed in $O(N \log N)$ time.

An OctTree can be viewed as a generalization of a Binary Search Tree (BST) with eight children instead of two. Recall that when traversing a BST, we compare the inserting node's key (a.key) with the current node's key (b.key). This comparison dictates which child we explore further. Traversing an OctTree follows identical semantics, but now each node has eight possible children and each node has three keys instead of one. The logic for determining which child branch to recurse onto, as well as a graphical representation of a QuadTree, is shown below.

```
C/C++  
// OctTree = Generalization of BST  
int idx = 0;  
if (body.x >= internal_node.x) idx |= 1;  
if (body.y >= internal_node.y) idx |= 2;  
if (body.z >= internal_node.z) idx |= 4;
```



Internal nodes in the OctTree represent an entire rectangular region of space that contains several bodies and leaf nodes represent individual bodies. The key operations we apply on the OctTree data structure are insert for tree construction and top-down traversal for force estimation.

The inputs to Barnes-Hut include:

- 1) N bodies where a body is defined by an initial location (x, y, z) , a mass m , and velocities (v_x, v_y, v_z)
- 2) Simulation time step dt
- 3) Force approximation threshold θ

The outputs of Barnes-Hut are the locations and velocities of the N bodies at the next time step. Barnes hut simulates the force of gravity by iterating many times steps. The outputs of the algorithm become the inputs at the next time step.

The Barnes-Hut computation is broken up into different phases that have sequential dependency on each other. The different stages are described below:

```
C/C++
// N := number of bodies
// 1) Compute box around all bodys
float length = compute_box(bodys);
// 2) Construct OctTree to divide regions
OctTreeNode *root = build_tree(bodys, length);
// 3) compute net forces on each body via tree traversal
traverse_tree(root, bodys, net_forces, theta);
// 4) apply update body position based on net force, velocity
update_points(bodys, velocitys, net_forces, dt);
```

compute_box is a reduction over all bodies to compute a cube length that contains all bodies.

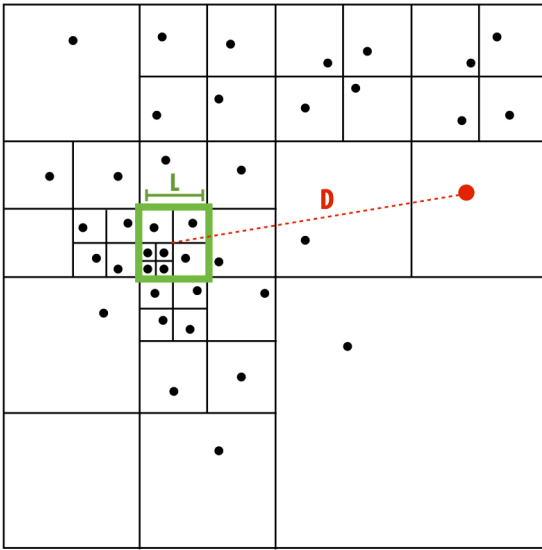
build_tree inserts the N bodies of interest into the OctTree in their corresponding region. Regions are further divided when the insertion location corresponds to a leaf node itself. To prevent OctTree depth explosion, the depth of the tree is capped and bodies at the max depth are treated as a single mass. As we traverse the OctTree searching for the correct leaf node to insert the new body, we update the center of mass of each internal node we visit to account for the newly inserted body.

traverse_tree traverses down the OctTree for each of the N bodies to accumulate the net gravitational force acting on that body. If we reach a leaf, we accumulate the gravitational force directly. If we reach internal node, we compute:

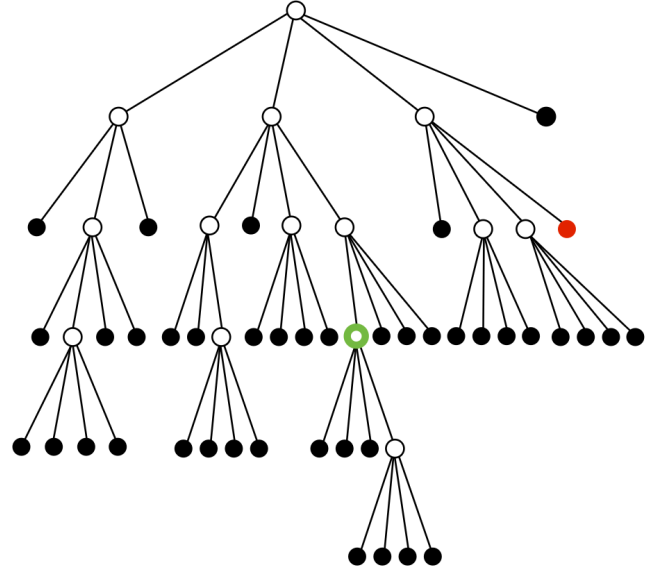
- 1) The distance D from the body of interest to the center of mass of bodies within the internal nodes region
- 2) The width of the region L of the current internal node

We use the threshold L/D to determine if we should approximate the force computation with the region via the region's center of mass. Regions that are farther away and are more tightly concentrated together are more likely to be approximated. If this threshold is small enough, we accumulate the gravitational force of the region and don't traverse any further. Otherwise, we continue traversal along all children nodes.

An example of how an OctTree is used in Barnes-Hut is illustrated in the diagram below. The red dot is the body we are computing net forces for. The green box represents an internal node in the OctTree we are considering approximating.



Spatial Domain



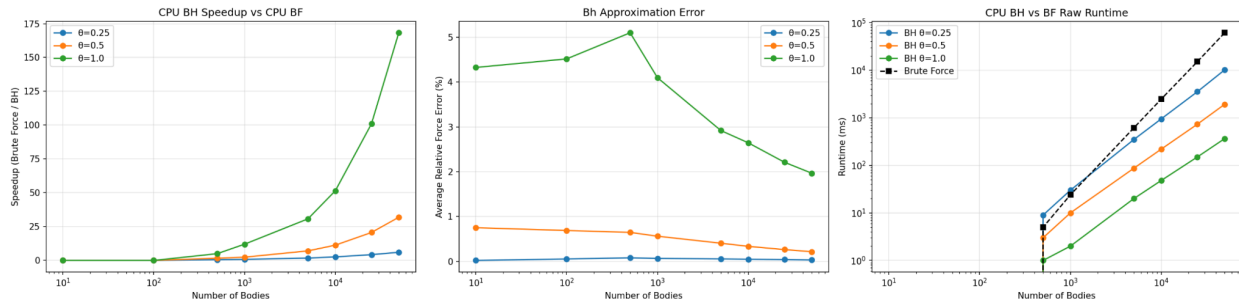
Quad-Tree Representation of Bodies

update_points applies a simple numerical update rule to change the body's position based on its current velocity and computed net force acting on it by other bodies.

CPU Implementation

The single-threaded implementation for Barnes-Hut on a CPU (CPU BH) follows the pseudocode given above. OctTree nodes are dynamically allocated onto the heap during **build_tree** and are freed from memory before the next time step. For all experimental results outlined in this report, I generate test cases by randomly sampling N bodies within a cube centered at the origin with side length of 10k. The mass of each body is sampled randomly between 50 and 100.

Below are results for CPU BH compared to a CPU Brute-Force (CPU BF) algorithm that performs exact gravity computation in $O(N^2)$ time.

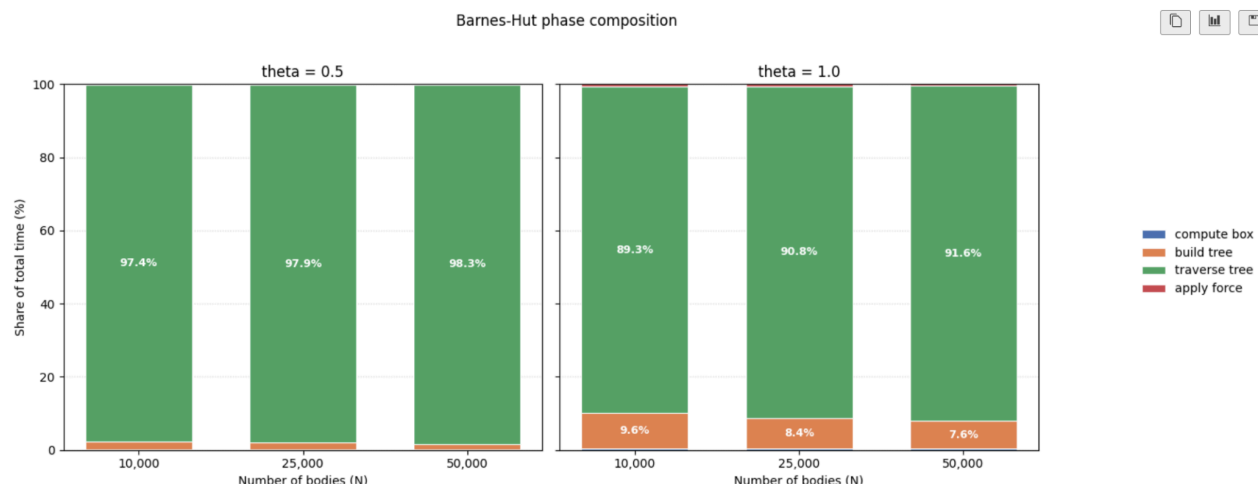


The 1st plot shows the benefits of estimating forces via OctTree traversal, where CPU BH obtains approximately a 175x speedup with 50k bodies for a single time step with $\theta=1$.

The 2nd plot shows the tradeoff between solution quality and simulation speed. The vertical axis shows the error between the estimated force computed by BH and the exact force computed by BF. As we expect, larger values of theta result in worse solution quality while obtaining much faster simulation, as we are more aggressive in pruning OctTree traversal during force computation. The small error in computed forces also acts as a correctness sanity check.

The 3rd plot shows raw runtime, highlighting how even at only 50k bodies, the runtime of CPU BF is already over one minute.

When analyzing the four phases for our sequential implementation, we see that ***traverse_tree*** dominates the overall runtime. This aligns with our algorithmic intuition: as computing the bounding box and updating points based on computed forces is $O(N)$, while traversal and tree construction are $O(N \log N)$. While traversal and construction are the same asymptotic complexity, each body inserted into the tree during construction only travels down a single path of the tree and nodes inserted early on in the construction algorithm are much shorter since the tree is growing dynamically. Traversal, on the other hand, can traverse many paths from the root and the tree, performs more computation per node, and all traversals work with the full size tree.



Parallelism Opportunities and Challenges in Barnes-Hut on GPU:

Bounding Box Computation: Requires 6 reductions over the min / max spatial coordinates and then 1 reduction over these optima to compute the length of a cube that contains all bodies.

Implementing this on a GPU will require less cleverness since classical reduction kernels can be applied for this computation.

Tree Construction: In theory, we are able to have several threads inserting into the OctTree and manipulating OctTreeNode data entries simultaneously so long as they are not inserting at the same edge link or writing updated data to the same node. Additionally, threads are able to

traverse the same internal node simultaneously if they are not manipulating any of the internal nodes data.

Maximizing these opportunities for parallelism is non-trivial. For example, in BH CPU, the center of mass for each internal node is updated on the fly as each body is being inserted into the tree. This logic would prevent two threads from traversing the same internal node simultaneously even if their desired insertion location is far down the tree.

Additionally, implementing a parallel tree construction strategy while leveraging the computational units of a GPU is non-trivial. Course-grained locking would serialize the algorithm and torpedo performance. Fine-grained + hand-over-hand locking would require locks for each node in the OctTree which does not scale well if we aspire to simulate N-body with millions of nodes.

Force Computation: The force applied on each body exhibits data parallelism: forces correspond to separate traversals of the tree and can be computed independently.

Implementing parallel tree traversal on a GPU comes with many challenges. First, recall that all threads within a warp execute in lockstep (they share an instruction via SIMD hardware). If some bodies finish their traversal long before others, the program may suffer from extreme warp divergence and result in those execution units will be sitting idle.

Second, if bodies assigned to threads in a given warp are far apart from each other in space, the warp traversal could touch all of the nodes of the OctTree. This poor data locality would require massive amounts of data to be fetched from global memory.

Force Application: Updates of each body location based on computed forces are embarrassingly parallel, similar to saxpy seen during the semester. Computed net forces, velocity and body location can be fetched from global memory and updated body location and velocity can be written back to global memory

GPU Implementation

To accelerate Barnes-Hut, I implemented it on a GPU using CUDA using ideas outlined in [1]. I first tried to implement an initial implementation of Barnes-Hut without referring to the paper at all. Ultimately, limited time prevented me from implementing an initial version with no reference to the paper due. My naive implementation was primarily bottlenecked by complexity in the OctTree representation on the GPU and the non-trivial synchronization logic during tree construction. This effort to implement is reflected in my first implementation (V1).

First, I will outline how the OctTree data structure is defined on the GPU.

Sequential OctTree GPU Representation

During ***build_tree*** in BH CPU, once the insertion location to a new node is found, the new OctTreeNode struct is allocated dynamically in heap memory. As a result, traversal of the OctTree requires frequent dereferencing of child pointers around heap memory.

While dynamic memory allocation is conceptually simple, it harms performance significantly on GPUs and can result in poor locality. As a result, [1] introduces a statically allocated array based OctTree representation that stores all relevant fields of data in separate linear arrays. These arrays are allocated once before the algorithm begins and remains resident on device memory over many timesteps.

To motivate their design, first recall that there are only two types of nodes in our OctTree: internal nodes, representing entire regions of bodies, and leaf nodes, representing individual bodies. There are 3 types of data fields: fields relevant for both Internal and Leaf nodes, for only internal, and for only leaf nodes. These fields fall into these categories as seen below:

A) Leaf Only

- Net Force / Velocity

B) Internal Only

- Children Pointers

AB) Internal + Leaf

- Center of Mass: Internal Nodes consider all bodies within their region, Leaf Nodes have a center of mass equal to their spatial coordinates
- Total Mass: Internal nodes have a total mass equal to the sum of the mass of their children, leaf nodes total mass is equal to their individual mass

To store these fields for each node, we pre-allocate separate fixed size arrays of length M where M is an upper-bound on the total number of nodes in the OctTree. The first N entries of these arrays correspond to information about the N bodies and entries from N to M correspond to internal nodes. Initially, the first N entries are populated with known data about body location, mass and initial velocity.

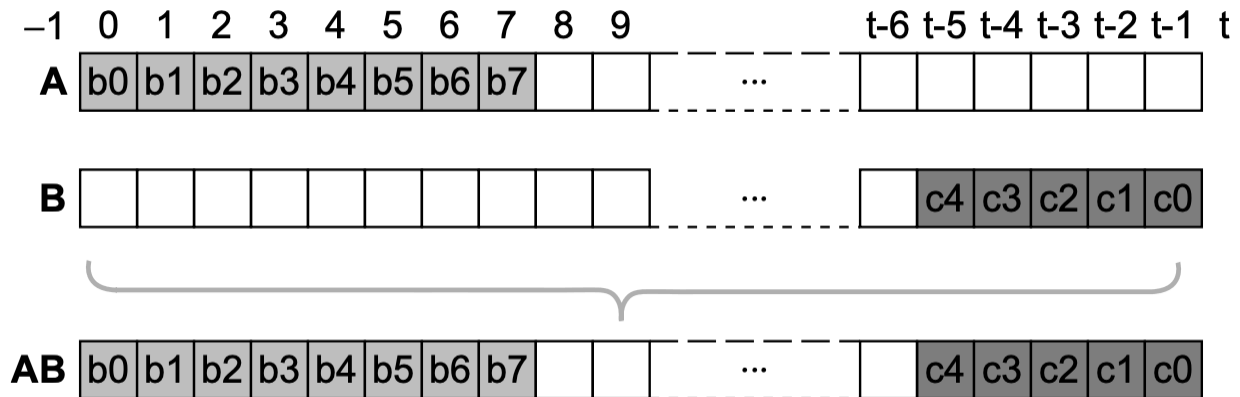
When constructing the OctTree, we replace invocation to a dynamic memory allocator (*new* / *malloc*) with a call to *atomic decrement*. First, an index variable *next_cell* is initialized to M . When a new OctTreeNode needs to be allocated during tree construction, a thread atomically grabs a new index by decrementing *next_cell*. The index returned by the atomic decrement is used to store all relevant information about this internal node across all pre-allocated data arrays.

This approach creates a mapping between the index space $\{-1, 0, 1, \dots, M-1\}$ to the M potential nodes in the OctTree, where

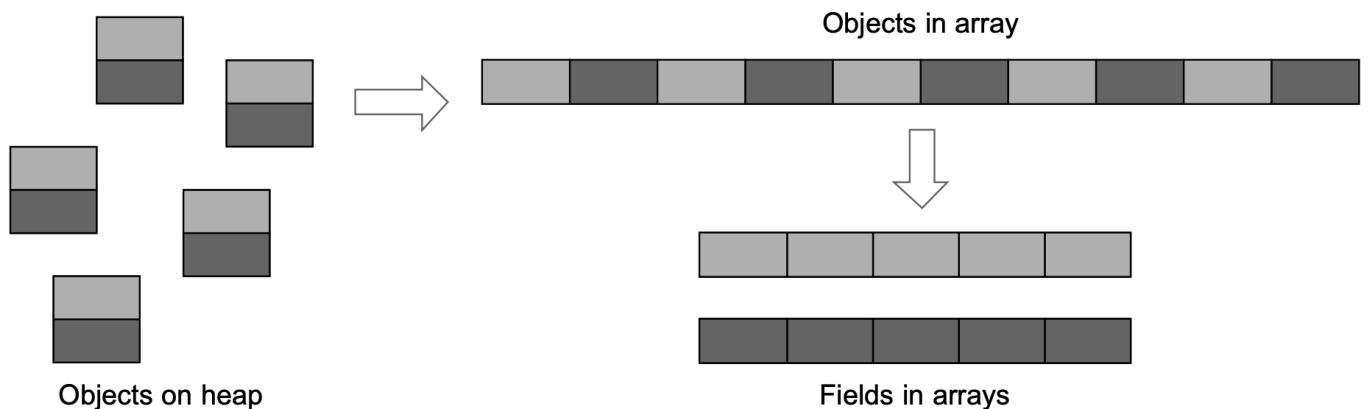
- -1 = Null

- 0:N-1 = Leaf Nodes
- N:M-1 = Internal Nodes

A diagram explaining the relationship between index space and data arrays can be seen in the diagram below. **A** represents a field where only body information needs to be stored (such as velocity). **B** represents a field where only internal node information needs to be stored (such as children). **AB** is a field where information is relevant for both types of nodes (location, mass)



One alternative static data representation for the OctTree is preallocating an array of OctTreeNode structs. One downside of this approach is that accesses to data fields cannot be coalesced together, since adjacent bytes in memory now correspond to different fields. For example, if all 32 threads in a warp attempted to fetch the mass of the body threadIdx, that memory accessed could not be coalesced and would result in 32 separate memory transactions. A comparison of these 3 approaches is illustrated in the figure below:



CUDA BH V1

Below is an overview of the kernels used in V1. The main algorithmic difference compared with BH CPU is that the construct tree has now been broken up into two separate kernels: one for constructing the parent/child relationships in the OctTree, and another for computing the center of mass of each internal node.

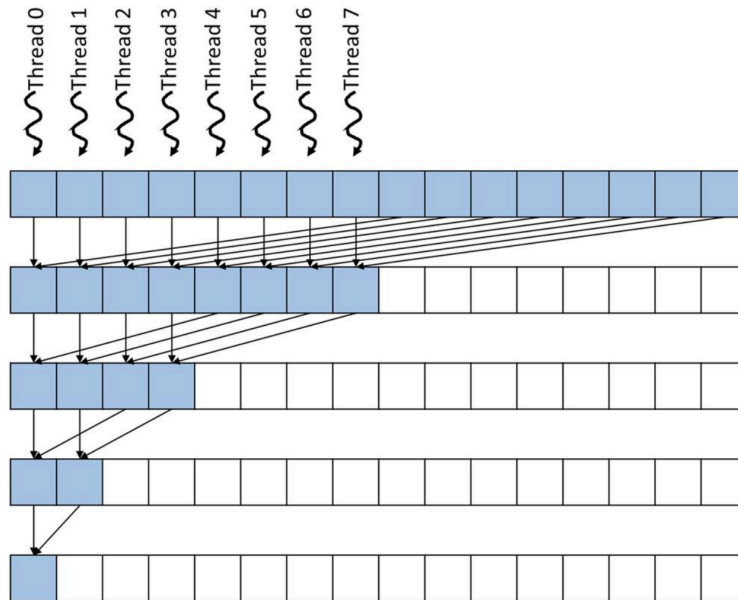
```
C/C++
// BLOCK & GRID SETUP: 1d grid and blocks for all kernels
// default: NUM_BLOCKS = SM_COUNT; BLOCK_SIZE = 4*WARP_SIZE
dim3 grid_dim(NUM_BLOCKS, 1, 1);
dim3 block_dim(BLOCK_SIZE, 1, 1);

//////// v1 cuda bh kernels //////////
// 1) reduction over body locations for bounding box
body_reduce_kernel<<<grid_dim, block_dim>>>(...);
// 2) construct sequential OctTree parent/child relationships
build_tree_kernelv1<<<grid_dim, block_dim>>>(...);
// 3) compute center of mass of internal OctTree Nodes
compute_cmass_kernelv1<<<grid_dim, block_dim>>>
// 4) compute force applied on each body via OctTree traversal
compute_forces_kernelv1<<<grid_dim, block_dim>>>(...)
// 5) update body locations via estimated forces
apply_forces_kernel<<<grid_dim, block_dim>>>(...)
```

body_reduce_kernel

Threads in each block are mapped to bodies and fetch x,y,z coordinates from global memory in a coalesced manner. Each thread computes a local min and max reduction over these bodies and writes these 6 reduces values into shared memory

Each block then performs a reduction over shared memory in a hierarchical manner. After this, each block has partially reduced values. The figure below shows how each thread block performs the hierarchical reduction by conditioning each round on a threads index.



An elected thread for each block then writes its corresponding reduced value to global memory. After it writes to global memory, the thread increments an atomic counter. If the returned value from the atomic counter equals *num_blocks-1*, this implies that all block-level reduction values have been written to global memory. This thread then performs the final sequential reduction by fetching all block-reduced values from global memory, computing the final result, and writing it back to global memory.

It's important to note that a memory fence is required after writing the block-level reduced values back to global memory. If we didn't include a memory fence after these global writes, it's possible that the final sequential reduction by the last elected thread could read global memory values before their updates were propagated to main memory.

C/C++

```
if (threadIdx.x == 0){
    blk_minx[blockIdx.x] = s_minx[0];
    // fence needed to ensure value is stored before the elected thread
    // performs global reduction across block values
    __threadfence();
    int old = atomicAdd(&blk_counter, 1);
    final_block = (old == NUM_BLOCKS-1);
}
if (final_block){
    // load blk_minx to reduce across blocks
}
```

build_tree_kernel

Threads in each block are mapped to bodies in the same way as in *body_reduce_kernel*. Each thread attempts to insert its assigned body into its correct location in the OctTree

One challenge with parallel tree construction is when multiple threads attempt to insert different nodes at the same location. As mentioned in the Parallelism Opportunities and Challenges section, there are several possible solutions to handle this:

- 1) The most naive implementation idea is course-grained locking - have one lock that must be acquired in order to traverse and insert into the OctTree. This would be catastrophic for performance and utilize essentially none of the GPU execution resources
- 2) Another implementation idea is fine-grained locking where each node contains its own lock. Traversal of the tree could then be executed via hand-over-hand locking. This implementation is not scalable since it requires a lock for every node in the OctTree, which will struggle for large input sizes. Additionally, it does not leverage the observation that simultaneous traversal of internal nodes of the tree preserves correctness so long as we don't manipulate that internal node's data.

To address the challenges of fine-grained hand-over-hand locking, I implemented a Compare-and-Swap based synchronization solution that requires locking a child node only when a thread inserts into the OctTree.

After a thread has traversed the OctTree and found the location to insert its corresponding body, it executes a Compare-and-Swap, attempting to write a designated LOCK value into the corresponding child array. This value informs all other threads traversing the tree that this location cannot be written to. Since we are using values stored in memory as flags for synchronization, we must ensure that the children array is volatile to guarantee that the thread does not cache a stale value and miss inserts from other threads. This logic is outlined below.

C/C++

```
// read child value from memory; if lock currently held, retry
int child = ((volatile int*)children)[node*8 + child_oct];
if (child == LOCK_VAL) continue
// attempt to acquire lock
int lock = atomicCAS(&children[node*8 + child_oct], childx, LOCK_VAL);
// if someone else was faster; need to retry
if (lock != childx) continue;
// LOCK OBTAINED AT THIS POINT //
```

Any thread that reads LOCK during its traversal is forced to restart the loop. The thread traversing the tree releases the lock by writing the index of the newly allocated node to the child array, over-writing the LOCK value. If the lock is obtained, the thread attempts to insert the body at this location (insertion logic identical to BH CPU)

Note that in the case where we need to recursively divide the region, we need to allocate several internal nodes in the tree before we release our lock. As a result, before overwriting the LOCK value in the child array, similar to the reduction kernel, a memory fence is required to ensure that all writes are visible to other processes before we release the lock. Without this memory fence, it's possible that the system would overwrite the LOCK value in the child array before other writes to the child array have propagated, resulting in another thread reading garbage values.

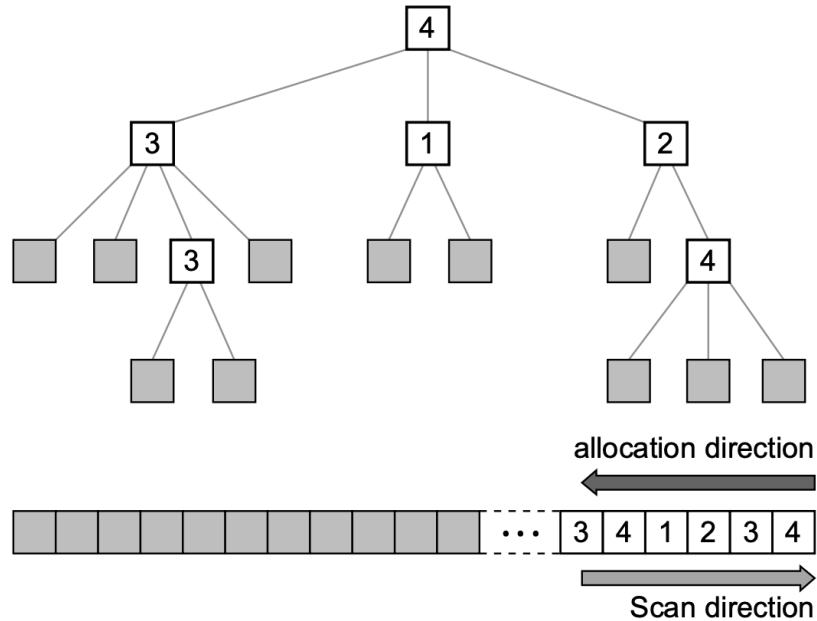
Kernel Performance Challenges:

- 1) There is very high contention on the Compare-and-Swap operation early on in tree construction when many threads are attempting to grab the same lock. Retries also put more pressure on the interconnect as the child value is fetched from global memory
- 2) There are large amounts of thread divergence in the construction of the tree; there is no known structure to the bodies being inserted within a warp. As a result, some bodies may require deep tree traversal while others require shallow traversal. Additionally, some threads may be successful in acquiring their lock while others are not, forcing the entire warp to stall

Ultimately, I didn't invest time into optimizing these challenges due to this kernel's smaller impact on overall runtime.

compute_cmass_kernelv1

Threads are mapped to internal OctTree nodes in a bottom up fashion. For each internal node, we want to compute its center of mass. I leverage the property that internal nodes are allocated in index space in decreasing order which implies that children nodes will have smaller indexes than parent nodes in the flat index space. See below for an illustration for how threads get mapped to internal nodes in the OctTree. Numbers represent mappings between internal nodes and threads



First, each thread first scans over its children:

- 1) If a child is null, skip
- 2) If the child is a body, update center of mass of internal node
- 3) If the child is also an internal node, we must ensure that this internal node has already computed its center of mass before we can incorporate it. We keep track of which children are not currently ready and we need to be waited on.

After doing an initial check on our children, we enter a do-while loop that periodically checks if any of our children that were previously not ready have been computed. We fetch from global memory to see if the mass is no longer -1, and if it is, we update our node's center of mass and decrement the number of children we are waiting for.

Once all children have been accounted for, we write the updated center of mass and total mass back to global memory, marking the node as ready to be processed by its parents.

Following the same pattern as in previous kernels, the center of mass must be stored to global memory before we store the mass since mass is acting as a flag for other threads. Thus, a memory fence is required between writing center of mass and writing mass to global memory

We achieve the ready/not ready logic by using the value of an internal node's mass to act as a flag to signal if the node is finished or not. All internal node's masses are initialized to -1, and when an internal node computes its center of mass from its children, it writes its total mass value, overwriting the -1 value, and signalling to its parents that it is ready to be incorporated into its parents center of mass.

As in the previous kernel, the volatile keywords for body data is critical for the correctness of this kernel to prevent the compiler from storing a stale mass value in a register for and causing a

forever while loop, or from storing a stale center of mass location and causing an incorrect value to be computed by the parent.

This kernel accomplishes correct waiting for internal nodes to be ready with no atomic operations!

compute_forces_kernelv1

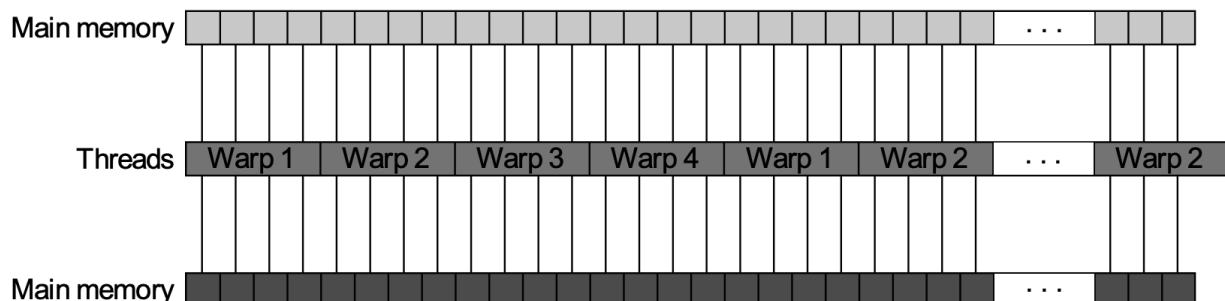
For this V1 implementation, I prioritized simplicity. Threads are mapped to bodies of which we need to compute the net force. Each thread fetches the center of mass and total mass of a body from global memory and performs a depth-first search on the OctTree to approximate the net force acting on its body. The traversal down the tree per thread follows the sequential implementation. Each thread keeps its own traversal stack in registers. After traversal has completed, we write the accumulated forces to global memory.

The primary challenges of this kernel include thread divergence, poor data locality + large global memory fetches and register overflow due to potentially deep traversal stacks.

apply_forces_kernel

Threads are mapped to bodies. In a streaming manner, we compute acceleration of each body by fetching the accumulated force and total mass for each body and update the body's velocity based on this acceleration.

We then update the body's location based on its velocity and write the updated body location to global memory. This kernels dynamics are outlined below



CUDA BH V1 Results

As an additional baseline, I implemented the $O(N^2)$ algorithm for gravity computation in CUDA (BF CUDA). Again, I launch a 1d grid of 1d thread blocks. Each thread in a block is mapped to a body. Each block then loops over all bodies with a stride equal to the block size. Each thread loads in their corresponding body into shared memory and then all threads in the block compute the gravitational force between their assigned body and the bodies loaded into shared memory. This drastically reduced the global memory accesses. An overview of BH CUDA is given below

C/C++

```
// default: NUM_BLOCKS = SM_COUNT; BLOCK_SIZE = 4*WARP_SIZE
dim3 grid_dim(NUM_BLOCKS, 1, 1);
dim3 block_dim(BLOCK_SIZE, 1, 1);

// directly compute  $O(N^2)$  law of physics leveraging shared mem
compute_force_bf<<<grid_dim, block_dim>>>(...);
// apply streaming kernel
apply_forces_kernel<<<grid_dim, block_dim>>>(...);
```

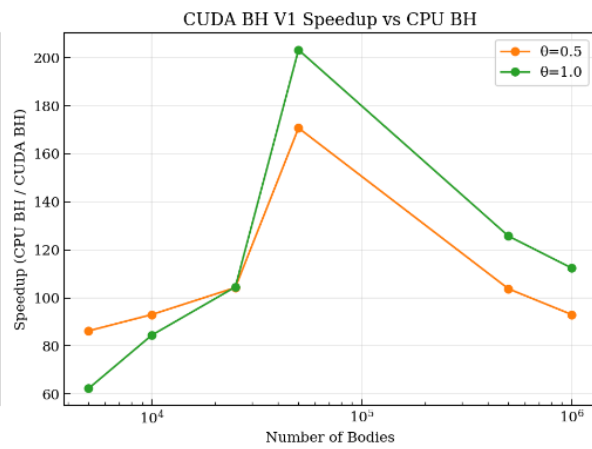
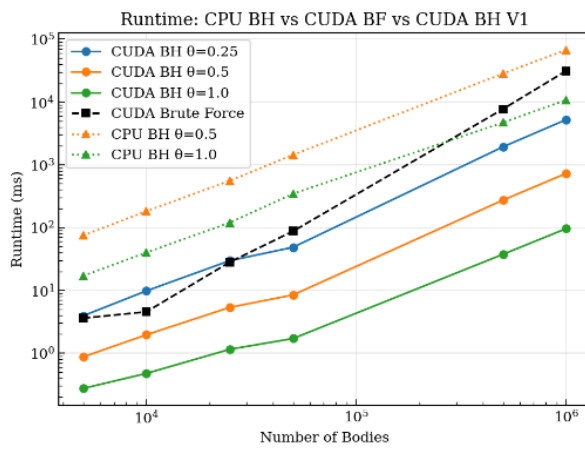
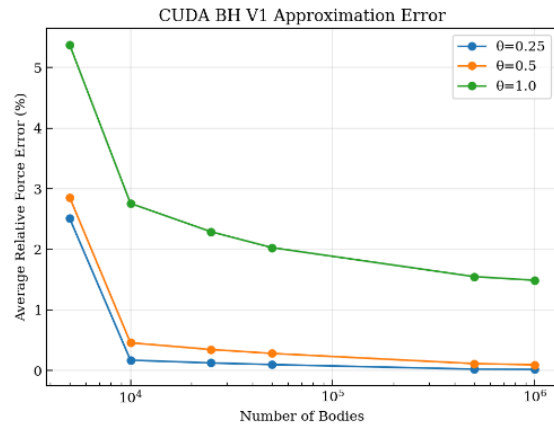
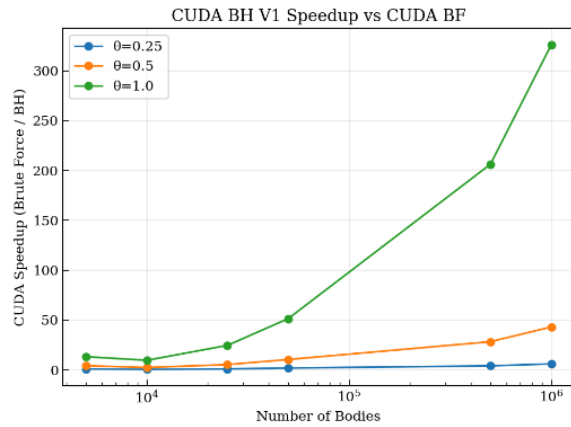
Below I report results for BH CUDA V1 compared to BF CUDA and BH CPU.

First, force approximation error is small and decaying with increased body count suggesting a correct implementation. See Appendix for more details on test cases used to ensure correctness of BH CUDA.

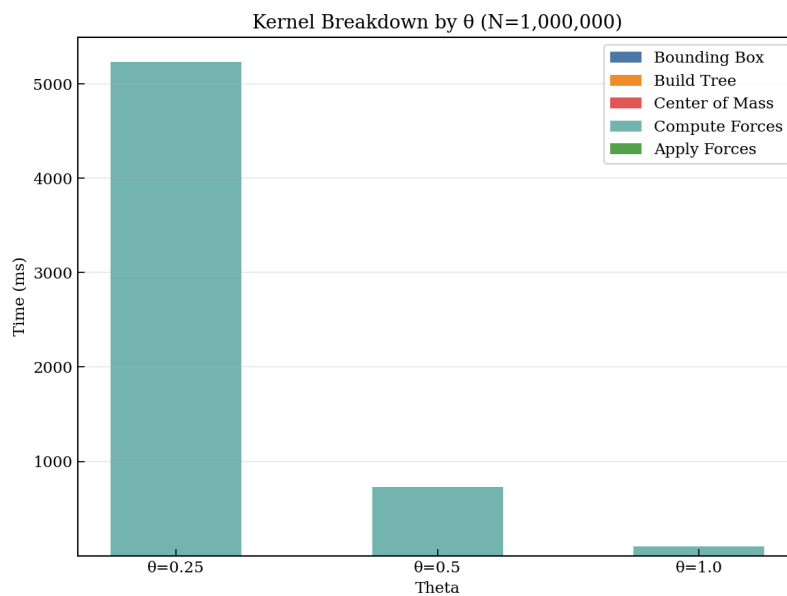
We observe a speedup compared to CUDA BF of over 250x with less than 2% error and 50x with less than 0.2% error. Recall that as θ goes to 0, CUDA BH should be strictly worse than CUDA BF since it has additional overhead and is computing the same force interactions. This explains the poor speedup when $\theta = 0.25$.

We observe between 80-110x speedup compared to CPU BH for body counts less than 50k but don't see improved speedup between BH CUDA vs BH CPU past 50k bodies. One speculation for why this occurs is because BH CUDA V1 implements force computation with separate threads in the warp having their own traversal stack. As N grows, the depth of the OctTree increases and more pressure is placed on registers. If the set of data being stored in registers exceeds the hardware capacity, these values will be dumped to global memory, drastically increasing the time to perform tree traversal.

Note that $\theta=0.25$ is excluded from these plots because of how slow execution is for 1 million bodies.



When analyzing each kernel for our V1 implementation, we see that force computation completely dominates the computation. As a result, our performance optimizations should target compute forces



V1 Profiling via Nsight

Given the above results that compute forces is the bottleneck of our implementation, I leveraged Nvidia's Profiling tools to collect more information CUDA BH V1.

Hypothesis: **compute_forces_kernelv1** has large amount of thread divergence which results in low GPU utilization and suboptimal performance. At its core, this divergence is due to the fact that each thread in the warp is assigned a different body in which to accumulate forces. Different threads will encounter different node types during their traversal, causing the internal/leaf conditions logic to both be executed. Additionally, some threads will terminate their traversal earlier than others causing that lane to be fully idle while others still traverse.

Measurement: I ran the following profile command to analyze the performance of **compute_forces_kernelv1** on 50k bodies. For the original Nsight outputs, see *results/profiles*

Shell

```
akouloge@ghc32:~/private/bhut$ ncu --kernel-name compute_forces_kernelv1  
--launch-count 1 --set detailed ./bhut_cuda
```

Measurement #1: Achieved Occupancy 12.42%.

I had set the number of blocks to be equal to the number of SMs on the GHC GPUs (Turning Architecture) which is 46 and the number of warps per block to be 4. According to Nvidia's Documentation [2], each SM in the Turing architecture has the capacity to hold 32 warps. By increasing the number of blocks we launch, we can increase the number of warps resident per SM. Therefore, if some warp is stalled on a global memory fetch, the hardware can swap it out for another warp which is ready to execute instructions.

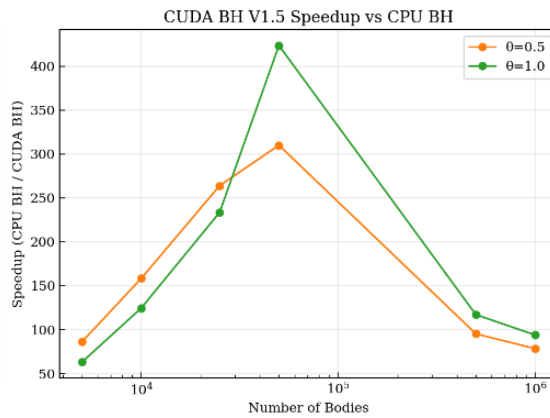
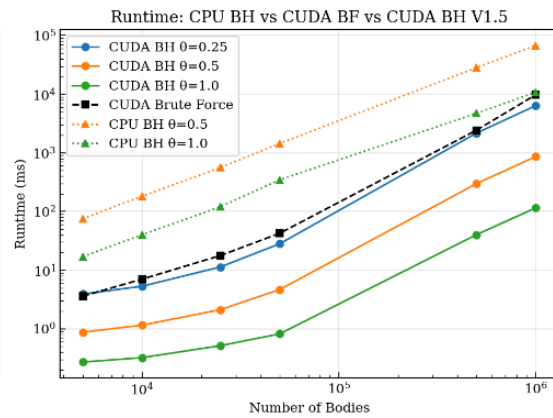
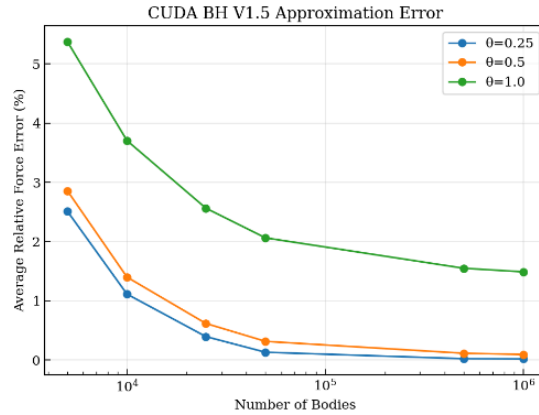
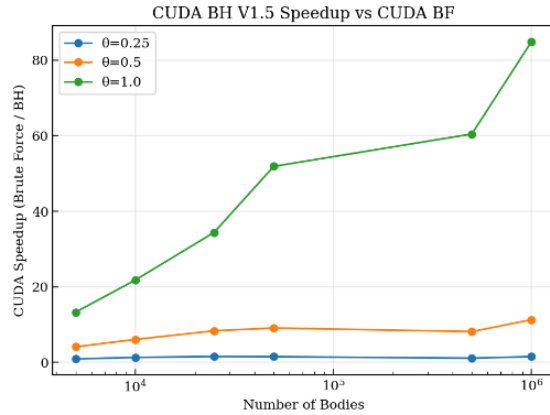
The code change I made was setting the number of blocks to cover the input of bodies. I applied this change to both CUDA BF and CUDA BH V1.

C/C++

```
OLD: dim3 grid_dim(46, 1, 1);  
NEW: dim3 forces_grid_dim((N + BLOCK_SIZE - 1) / BLOCK_SIZE, 1, 1);  
// update force computation for both bhut and brute force  
compute_forces_kernelv1<<<forces_grid_dim, forces_block_dim>>>(...);  
compute_force_bf<<<forces_grid_dim, forces_block_dim>>>(...);
```

After rerunning the profiler, I see that achieved occupancy has now improved for this kernel from **12% → 87%**. See below for the updated plots. We denote this updated implementation with a larger grid as BH CUDA V1.5

Note that now that we have a stronger BF CUDA baseline, the relative speedup has decreased, while the overall performance of BH CUDA has improved.



Measurement #2: Average Active Threads per Warp: 11/32

This confirms my hypothesis and conceptual understanding.

Key Measurement #3: No Eligible = 80%

Every clock cycle, the warp scheduler selects a warp to execute an instruction. If no warps on the SM are eligible to be run, this instruction is marked as "No Eligible" (basically marking this clock cycle as stalled; indicating that our algorithm is doing a poor job at hiding latency [3]).

Since there are no synchronization operations in our force computation kernels, this metric is suggesting that our force computation kernel is memory bound. By isolating profile analysis on **`compute_forces_kernelv1`**, which dominates the runtime of our program, I was able to confirm this hypothesis via kernel level profiling. Recall the hardware specs for the GPUs on the GHC cluster:

NVIDIA GeForce RTX 2080 specs:

- Peak DRAM Bandwidth: 448GB/s
- Peak Fp32 FLOPs: 10,000 GFLOP/s

As a result of this, the Arithmetic Intensity Ridge is approximately 22.32 Flop / Byte.

To query information about the compute force kernel, I measured the number of fused floating point operations and FP32 operations to get a proxy for total FLOPs as well as the total number

of bytes fetched from DRAM during the program's execution. I profiled the compute force kernel on a trace with N = 500k

```
C/C++
ncu --kernel-name <kernel_name> --metrics
sm__sass_thread_inst_executed_op_ffma_pred_on.sum,dram__bytes.sum,dram__
bytes.sum.per_second,gpu__time_duration.sum,sm__sass_thread_inst_executed_op_fp32_
pred_on.sum ./bhut_cuda > <output_file>.txt
```

As the table below illustrates, the arithmetic intensity of Barnes-Hut V1.5 is extremely low.

CUDA BH V1.5

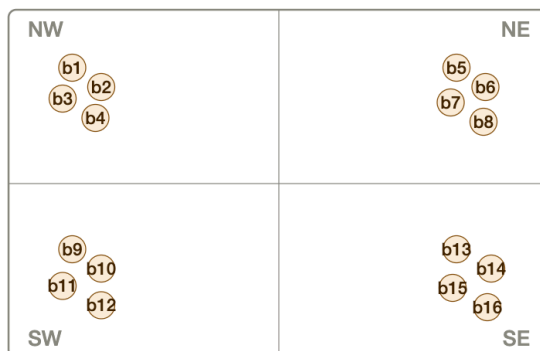
Kernels	GFLOPs	GFLOPs / sec	GBytes	GBytes / sec	Arithmetic Intensity	time (sec)
compute_forces_kernelv1.5	45	146	57.89	187	0.24	.308

CUDA BH V2

The above analysis confirms that ***compute_forces*** is the bottleneck for our programs performance and it is highly memory bound (low arithmetic intensity). My hypothesis is that this is caused by excessive global memory fetches during tree traversal. The root cause of these global memory fetches is the poor locality among bodies within a given warp: if bodies in a warp are far apart in space, the warps OctTree traversal will require a large set of data to be fetched to compute forces. An example of this is illustrated in the figure below.

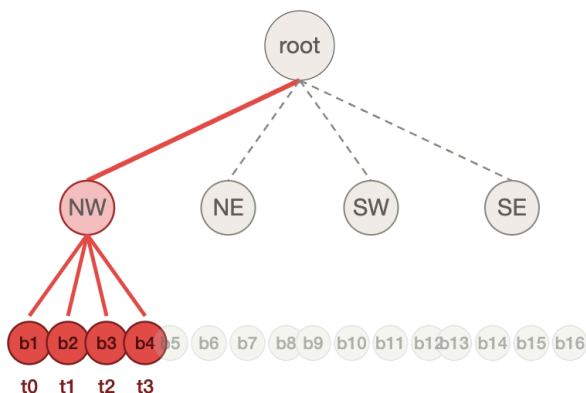
If bodies in warp are in different parts of the tree, traversal will consider a large part of the tree and will require fetching data from every tree node from memory, resulting in poor data locality (see tree on right). If bodies are very close together, then part of the tree that is traversed and the set of data that must be fetched is much smaller. Additionally, the data that is fetched can be shared among all threads in the warp (see tree on left)

Spatial layout: 16 bodies clustered by quadrant



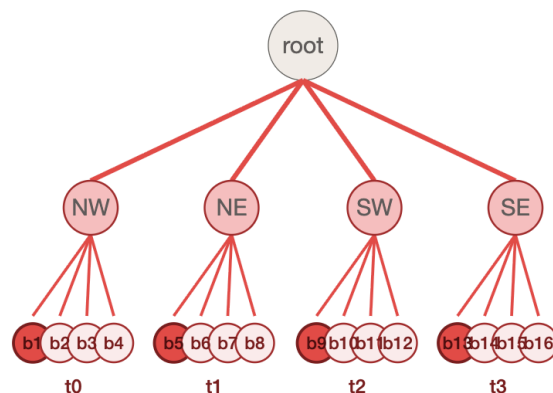
Clustered warp: {b1, b2, b3, b4}

All 4 bodies live in NW



Scattered warp: {b1, b5, b9, b13}

One body per quadrant



To improve the locality of data accesses performed by threads and reduce thread-divergence during force computation, we introduce an additional kernel:

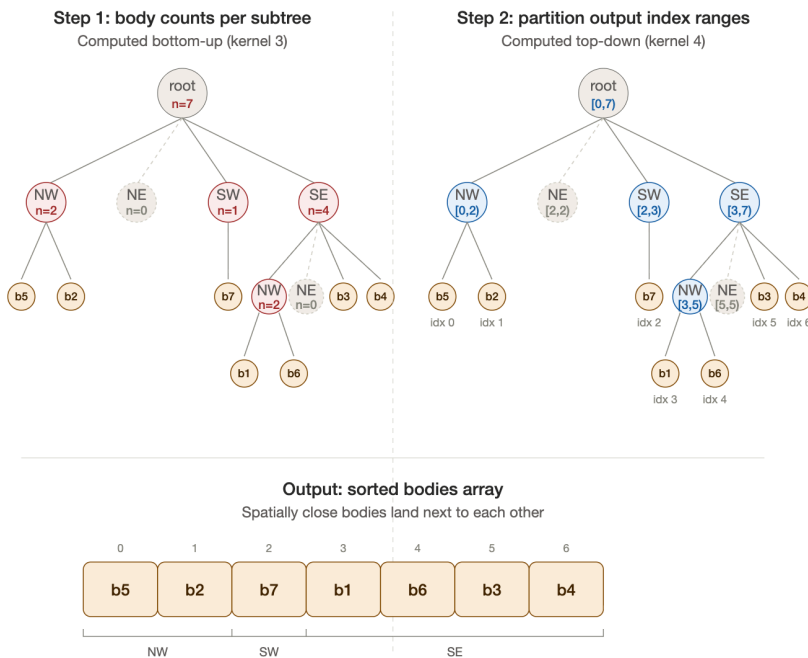
top_down_body_sort_kernel The purpose of this kernel is to produce an array in which bodies are ordered according to an in-order traversal of the octree. Because the octree is spatially decomposed, an in-order traversal groups spatially nearby bodies at nearby array indices. This sorted ordering is the key enabler for improving data locality in the force-calculation kernel: when the force kernel assigns bodies to threads, the 32 threads in a warp end up processing spatially clustered bodies whose tree traversals visit largely the same nodes. The per-warp traversal therefore stays converged, and the memory accesses are substantially reduced.

The sort is computed via a parallel top-down traversal of the octree. Each internal cell needs to know two things before it can be processed: (1) the total number of bodies in each of its children's subtrees, and (2) the offset at which its own bodies should begin. The first quantity can be computed during ***compute_cmass_kernelv1*** with very little overhead cost since that

kernel is already performing a bottom up traversal of the internal nodes. The second quantity must be propagated from the root downward.

Threads are assigned to internal nodes in a top down manner. Similar to the synchronization mechanism used in ***compute_cmass_kernelv1***, each thread waits for its parent node to have finished computing its offset in the sorted bodies array. After the node is marked as ready, we loop over the children and aggregate the body count in their respective subtree with the offset passed down by our parent. If a child node is a leaf, we write that body's index into the sorted array at the corresponding offset. In between processing children nodes, we perform the accumulation of their subtree body count into our current offset.

The figure below illustrates how the algorithm operates on a quad tree:



Equipped with a sorting of leaf nodes, we make a few changes to compute forces:

Warp cooperative traversal: Threads no longer perform traversal individually; traversal is performed at the warp level. While GPU hardware forces each thread in a warp to execute in lock step, and masking is applied when threads in a the same warp attempt to execute different instructions, our new traversal algorithm structurally enforces each thread in a given warp to act together.

If the node in the traversal is a leaf, then all threads incorporate its gravitational effect directly. If a node is an internal node, all threads compute their threshold value with respect to this internal node. Each thread in the warp then performs a warp-level voting primitive. If all threads agree to approximate the gravitational force from this cell, then all threads in the warp end their traversal of this branch. If at least one thread does not want to approximate, all 32 threads will traverse the children of this internal node. Note that this requires some threads to do strictly more work than necessary. Due to the nature of SIMD execution on GPUs, there is no point in having some

threads in the warp be idle while other threads in the warp traverse deeper in the tree. By having these threads traverse deeper, we can improve the accuracy of their computation without incurring additional runtime.

Elected-thread loads node data into shared memory. Since all threads in the same warp traverse the tree together, instead of making 32 requests to global memory for the current nodes data, we elect one thread to load in the nodes data from global memory into shared memory. Then each thread in the warp reads from shared memory to compute thresholds and gravitational forces. This greatly reduces the number of global memory accesses per warp traversal.

Warp-level traversal stack. We place the warp-level traversal stack in shared memory which takes pressure off of storing the traversal stack in registers for each thread.

The outline of BH GPU V2 is shown below:

```
C/C++
body_reduce_kernel<<<grid_dim, block_dim>>>(...);
build_tree_kernel<<<grid_dim, block_dim>>>(...);

// slight change from v1: now captures sub-tree body count
compute_cmass_kernelv2<<<grid_dim, block_dim>>>(...);

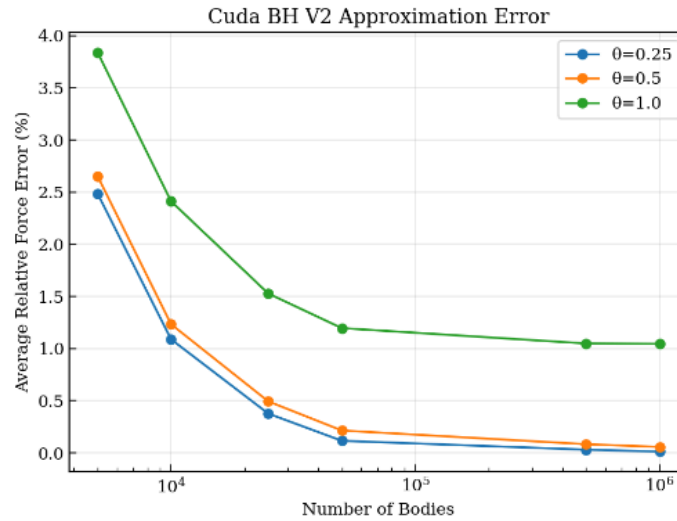
// NEW: create a sorted in order traversal list of bodies
top_down_body_sort_kernel<<<grid_dim, block_dim>>>(...);

// NEW: warp level traversal, elected thread loads GMEM -> SMEM,
compute_forces_kernelv2<<<forces_grid_dim, forces_block_dim>>>();

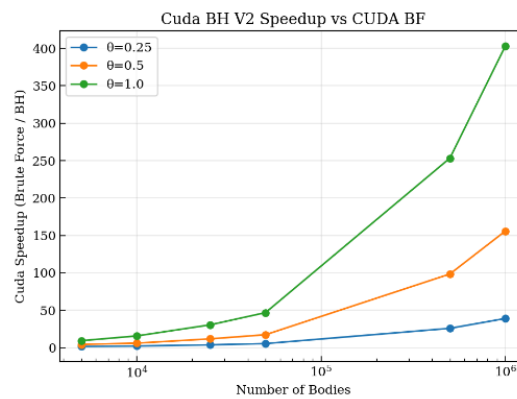
apply_forces_kernel<<<grid_dim, block_dim>>>(...);
```

CUDA BH V2 Results

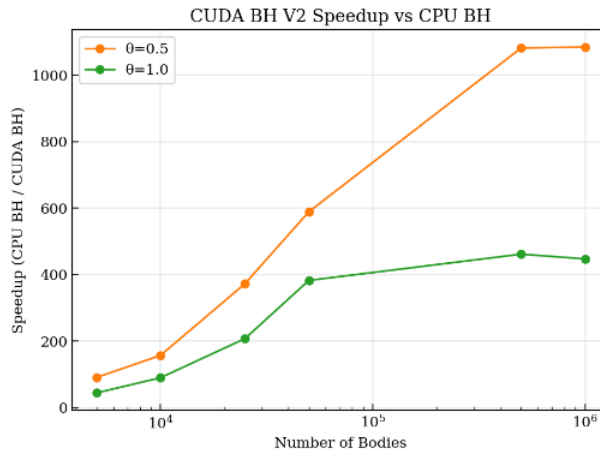
Approximation error is ~0.5% lower for $\theta=1.0$ for CUDA BH V2 compared to CUDA BH V1. This is expected due to the warp-level traversal improving solution quality:



Compared to CUDA BF, the v2 speedup for $\theta=1.0$ is over 400x compared to 80 in CUDA BH V1.5



Compared to CPU BH, we no longer see the diminished scaling results for body counts larger than 50k. For 1 million bodies, we see over a 700x and 180x speedup compared to barnes-hut with $\theta=0.5$ and $\theta=1$ respectively. Note that for V1.5, this was 100x and 90x respectively. Do note that the speedup diminishes at 1 million bodies. My hypothesis is that if we generated larger test cases, we would see a similar trend as we did for V1 and V1.5.



CUDA BH V2 Profiling via Nsight

Measurement: I ran the following profile command to analyze the performance of ***compute_forces_kernelv2*** on 50k bodies (for continuity to compare with v1)

Shell

```
akouloge@ghc32:~/private/bhut$ ncu --kernel-name compute_forces_kernelv2
--launch-count 1 --set detailed ./bhut_cuda
```

Measurement #1: Average Active Threads per Warp: (V1.5 = 11/32 → v2 = 28.4/32)

Expected due to warp-level tree traversal

Measurement #2: No Eligible (V1.5 = 80% → V2 = 38.76%)

Warps are now idle half as often as they were in CUDA BH V1.5, suggesting that the improvement to data locality worked in reducing stalls due to memory fetches

Measurement #3: L1 Cache Hit Rate (V1.5 = 12% → v2 = 32%)

More evidence of the improved locality of the V2 traversal. Now that warps within the same block are assigned bodies that are spatially near each other, they are more likely to touch overlapping portions of the OctTree.

I also analyzed the per-kernel characteristics of CUDA BH V2 and compared it with CUDA BF.

CUDA BF

Kernels	GFLOP	GFLOP / sec	GBytes	GBytes / sec	Arithmetic Intensity	time (sec)
compute_force_bf	17,750	6431	0.665	.241	26,691	2.76
apply_forces_kernel	0.025	0.9	0.31	382.25	0.08	~0
Total	17,750	6431	0.98	0.355	18,112	2.76

The CUDA BF implementation is heavily compute bound due to leverage shared memory to amortize global memory fetches across threads in the same block.

CUDA BH V2

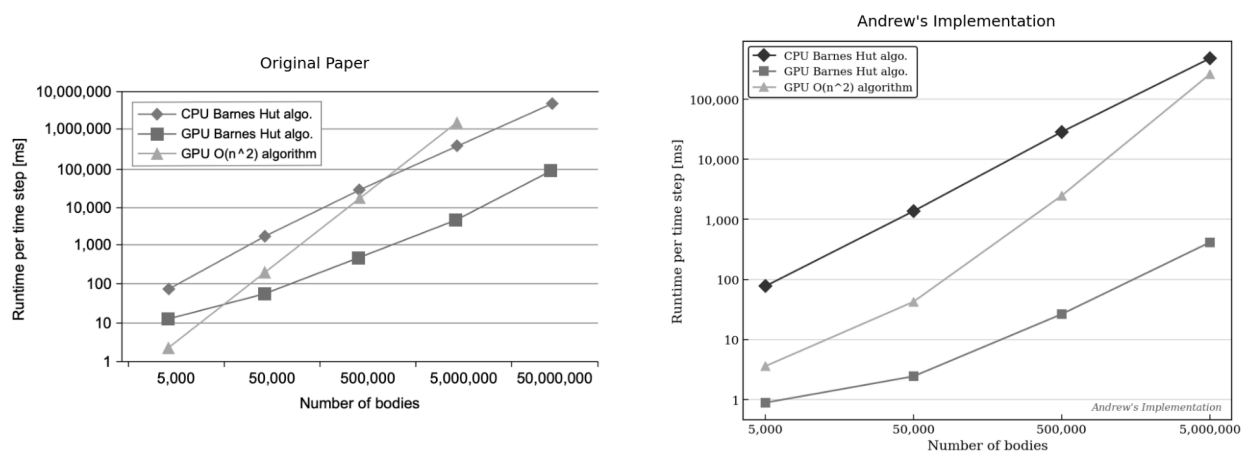
Kernels	GFLOPs	GFLOPs / sec	GBytes	GBytes / sec	Arithmetic Intensity	time (sec)
body_reduce_kernel	0.003	1.3	0.007	306.47	0.42	~0
build_tree_kernelv2	0.033		0.054			~0.001
compute_cmass_kernelv2	0.022		0.185			~0.002
top_down_body_sort_kernel	0	-	0.018		0	~0
compute_forces_kernelv2	90	3600	2.83	111	32.43	0.025
apply_forces_kernel	0.025	0.9	0.31	382.25	0.08	~0
Total	90.1		3.3		30	0.027

	GFLOPs	GFLOPs / sec	GBytes	GBytes / sec	Arithmetic Intensity	time (sec)
compute_forces_kernelv1	45	146	57.89	187	0.24	.308
compute_forces_kernelv2	90	3600	2.83	111	32.43	0.025

Conclusion

See below for my replication of the main results figure given in [1] comparing CPU BH, CUDA BF and CUDA BH. First, note that our CPU BH implementation has nearly identical performance

suggesting robustness of my sequential implementation (which was written from scratch). For the GPU implementation, we can see that the raw performance of my implementation is significantly faster than the results laid out in [1]. My hypothesis for this is due to hardware. Their implementation is run on 1.3 GHz Quadro FX 5800 GPU with only 30 SMs while mine runs on a NVIDIA GeForce RTX 2080 with 46 SMs. My hypothesis is that having 16 additional SMs would be massively impactful on the runtime of the CUDA implementation since the bottleneck of the Barnes-Hut program is the force computation step which is data parallel. Other spec improvements such as cache sizes, HBM bandwidth and clock rate would also contribute. The ability to be computing more bodies simultaneously would result in direct performance improvement. Better hardware also would explain why CUDA BF performs better than CPU BH even up to 5 million bodies in my experiments but is slower than CPU BH in the experiments conducted in [1]



N	GPU BF / GPU BH Speedup	CPU BH / GPU BH Speedup
5,000	4x	77x
50,000	11x	341x
500,000	53x	618x
5,000,000	423x	794x

Future Directions

- 1) Quantify why we see decreasing speedup in CUDA BH V1 implementation compared to CPU BH past 50k examples. The working hypothesis is primarily due to register spillage
- 2) Perform an ablation study on how different number of blocks affects the performance of **compute_forces**

References:

- [Barnes-Hut CUDA Paper](#) [1]
- [Nvidia Nsite Tutorial](#) [2, 3]
- [Princeton N body assignment](#)
- [Princeton Barnes-Hut assignment](#)
- [Barnes-Hut wiki](#)
- [PMPP](#)
- [15-418 course material](#)